Assignment Title

CA1 - Docker Tutorial Complete Guide

Student Name
Fady Abdelrahman Mohamed

Student ID
20053593

Lecturer:
Paul Laird

Module/Subject Title
Cloud Infrastructure & Virtualisation(B8IS003)

Submission Date: 23 March 2026
Academic Year 2025/2026

Acknowledgements & Aspirations

This project represents a humble step toward a future where technology and creativity converge. I believe that every line of code & report written today, and every piece of knowledge gained from **my lecturers and those around me**, is a cornerstone for tomorrow's success.

This work is deeply dedicated to my mother, who bought me my very first computer when I was only 11 years old, at a time when there were many other priorities more important than that. Her sacrifice and vision started this journey. I also dedicate this to everyone who believed in my potential and never stopped supporting me to turn visionary ideas into tangible reality.

The Quote:

"THE FUTURE BELONGS TO THOSE WHO BELIEVE IN THE BEAUTY OF THEIR DREAMS." Eleanor Roosevelt.

Declaration by Students

I, Fady Abdelrahman Mohamed (Student ID: 20053593), hereby declare that the work presented in this report is my original work and has not been previously submitted for academic assessment at this or any other institution. All literature, data, and work done by others and cited within this assignment has been given due acknowledgement and listed in the References section.

Signature: Fady Abdelrahman Mohamed
Date: 25 March 2026

Contents

Executive Summary 8

Key Achievements 8

Introduction 9

Assignment Objective 9

Why This Assignment Matters 9

Learning Environment 9

What ◦ Did - Overview 9

Simple Explanation 9

Technical Overview 10

Part 1: Getting Started with Docker 10

What I Did 10

Step 1: Installed Docker on Ubuntu 10

Step 2: Ran My First Container 11

Step 3: Started the Docker Tutorial Container 12

Part 2: Building My First Container 14

What I Did 14

Step 1: Downloaded the Sample Application 14

What I Got 14

Step 2: Created a Dockerfile 14

What Each Line Means 15

Step 3: Built the Docker Image 15

Explanation 15

What Happened: 16

Step 4: Verified the Image 17

Step 5: Ran the Container 17

Explanation 17

Step 6: Verified Container Status 17

Part 3: Updating the Application 18

What I Did 18

Step 1: Modified the Source Code 18

Step 2: Rebuilt the Image 18

Step 3: Removed the Old Container 19

Step 4: Started the Updated Container 19

Part 4: Sharing via Docker Hub 20
What I Did 20
Step 1: Created Docker Hub Account 20
Step 2: Created a Repository 21
Step 3: Logged In to Docker Hub 21
What I Entered 21
Step 4: Tagged the Image 21
Explanation 21
Step 5: Pushed to Docker Hub 22
What Happened 22
Step 6: Verified on Docker Hub 22
Part 5: Data Persistence with Volumes 23
The Problem: Data Loss 23
Step 1: Demonstrated Data Loss 23
Why This Happened: 24
The Solution: Docker Volumes 24
Step 2: Created a Docker Volume 24
Step 3: Ran Container with Volume 24
Explanation 25
Step 4: Tested Data Persistence 25
What I Did: 25
Part 6: Multi-Container Applications (Most Important) 26
Understanding the Architecture 26
The Scenario: 26
Why Separate Containers? 26
Architecture Diagram: 27
Manual Multi-Container Setup 27
Step 1: Created a Docker Network 27
Step 2: Started MySQL Container 27
Explanation 28
Step 3: Verified Both Containers Running 28
Step 4: Connected to MySQL 28
Step 5: Connected App to MySQL 29
Explanation: 29

Step 6: Verified App Uses MySQL 30
Docker Compose (Automated Multi-Container) 31
The Problem with Manual Setup 31
The Solution: Docker Compose 31
Step 7: Created docker-compose.yml 32
File Content: 32
What This Defines 33
Step 8: Stopped Manual Containers 34
Step 9: Started with Docker Compose 34
Step 10: Verified Compose Deployment 35
Step 11: Tested the Application 35
Step 12: Viewed Logs 35
Step 13: Stopped and Restarted 35
Conclusion 36
Key Achievements: 36
References 36

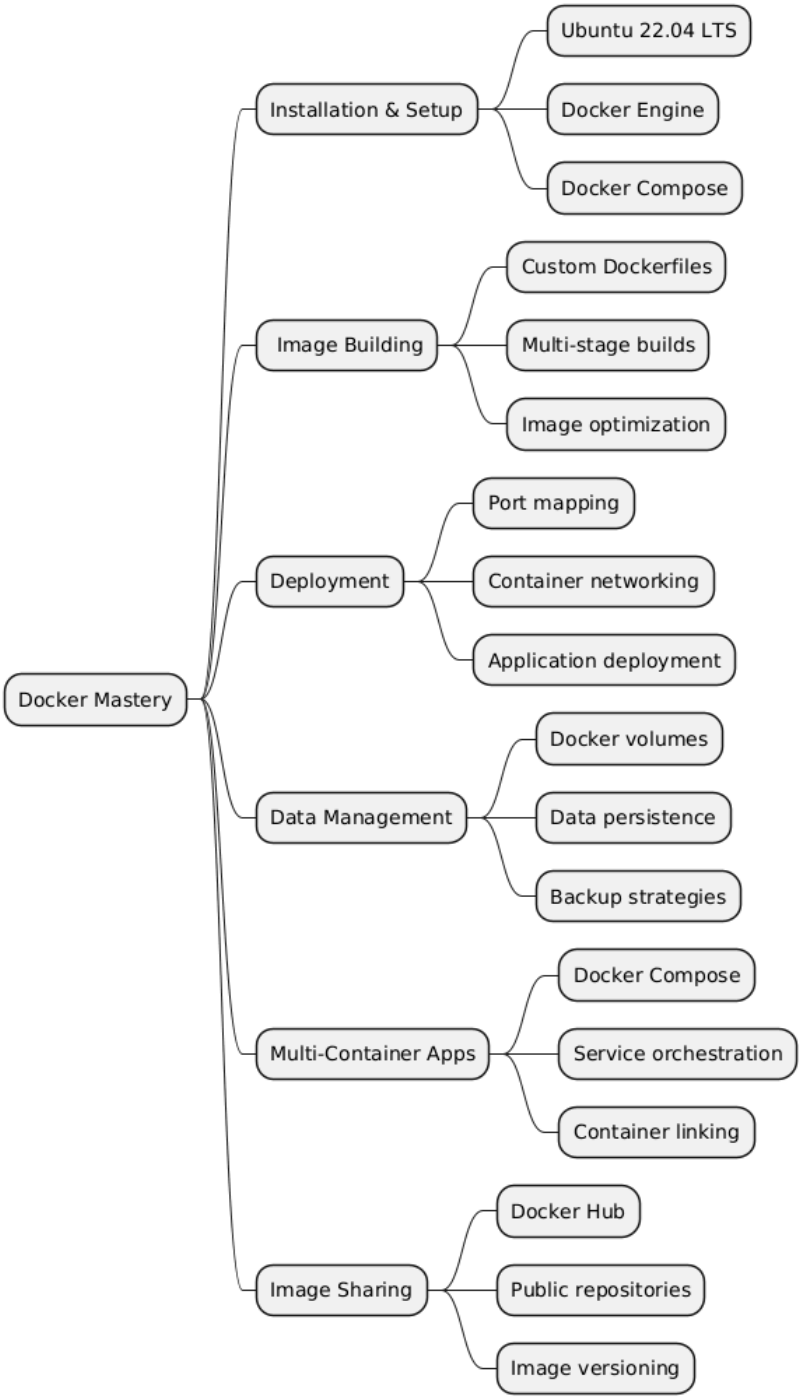
Figure 1Ubuntu 22.04.x LTS 10
Figure 2 Docker Version Output 11
Figure 3 Confirmed Docker was working properly 12
Figure 4 getting-started-browser ..... **Error! Bookmark not defined.**
Figure 5 Getting-Started-Browser 13
Figure 6 Tutorial Homepage 13
Figure 7 Container-Running ..... **Error! Bookmark not defined.**
Figure 8 App Folder Structure 14
Figure 9 Dockerfile Created 15
Figure 10 Image List 17
Figure 11 Todo App Running 17
Figure 12 Container Running 18
Figure 13 Code Modified 18
Figure 14 Updated App 19
Figure 15 Docker Hub Dashboard 20
Figure 16 Docker Login 21
Figure 17 Image successfully pushed to Docker Hub. 22
Figure 18 Todos Before Restart Todos Lost After Restart 24
Figure 19 Data Persisted 26
Figure 20 Multi-Container Architecture 27
Figure 21 Network Created 27
Figure 22 Terminal showing both containers running ..... **Error! Bookmark not defined.**
Figure 23 MySQL Connected 29

Figure 24 App with MySQL 29
Figure 25 MySQL - User Journey 30
Figure 26 Browser with Todos MySQL Data added 30
Figure 27 MYSQL DATA 31
Figure 28 Compose File 33
Figure 29 Compose PS 35
Figure 30 Compose App Running 36

Executive Summary

This report documents my completion of the official Docker "Getting Started" tutorial for CA. I followed the Docker tutorial step-by-step, learning how to containerize applications, manage data persistence, and deploy multi-container applications using Docker Compose.

Key Achievements



Introduction

Assignment Objective

The objective of CA1 was to follow the official Docker "Getting Started" tutorial from beginning to end, documenting each step with screenshots and explanations. The assignment required me to complete the tutorial up to and including the deployment of multiple containers using Docker Compose.

Why This Assignment Matters

This assignment teaches fundamental Docker skills that are essential for modern cloud infrastructure. Docker is an industry-standard tool for:

- Containerization: Packaging applications with all their dependencies
- Portability: Running applications consistently across different environments
- Efficiency: Using fewer resources than traditional virtual machines
- Scalability: Easily scaling applications up or down

Learning Environment

ᵒ completed this assignment using:

- **Operating System:** Ubuntu 22.04 LTS (running on Google Cloud VM)
- **Docker Version:** Docker Engine 24.x
- **Docker Compose Version:** v2.x
- **Application:** Node.js todo list application (provided by Docker tutorial)

What ᵒ Did - Overview

Simple Explanation

In plain language, here's what I accomplished:

1. Installed Docker - I set up Docker on my Ubuntu server, which is like installing a tool that lets me run applications in isolated "containers"
2. Built a Container - I took a sample todo list application and packaged it into a container using a recipe file called a "Dockerfile"
3. Made it Accessible - I configured the container so I could access the application through a web browser using port 3000
4. Shared it Online - I uploaded my container image to Docker Hub (like GitHub, but for containers) so others can download and use it
5. Added Data Storage - I attached a "volume" to the container so that data (todo items) wouldn't disappear when I restart the container
6. Created Multi-Container Setup - I connected two containers together: one running the application and another running a MySQL database, using Docker Compose to manage both

Technical Overview

From a technical perspective, I completed the following phases:

Phase	Technical Task	Purpose
Phase 1	Docker installation and verification	Establish containerization environment
Phase 2	Dockerfile creation and image building	Learn container image creation
Phase 3	Container deployment with port mapping	Enable external application access
Phase 4	Image sharing via Docker Hub	Demonstrate image distribution
Phase 5	Volume implementation for data persistence	Solve ephemeral container data problem
Phase 6	Multi-container orchestration with Compose	Deploy production-like architecture

Part 1: Getting Started with Docker

What I Did

Step 1: Installed Docker on Ubuntu

I installed Docker Engine on my Ubuntu 22.04 LTS server using the official installation commands.

Commands Used:

```
# Update package index
sudo apt-get update

# Install Docker
sudo apt-get install docker.io -y
```

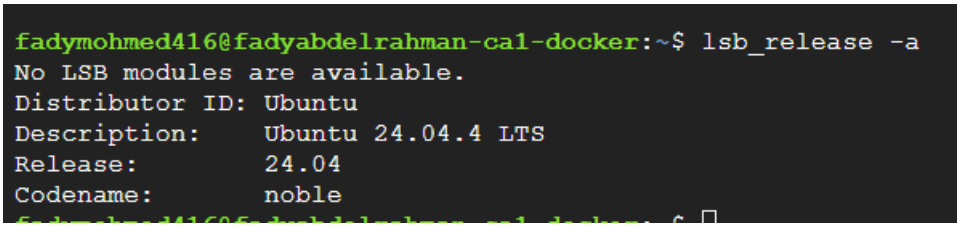


Figure 1Ubuntu 22.04.x LTS

 # Verify installation
docker --version
docker compose version

```
fadymohmed416@fadyabdelrahman-ca1-docker:~$ docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
```

Figure 2 Docker Version Output

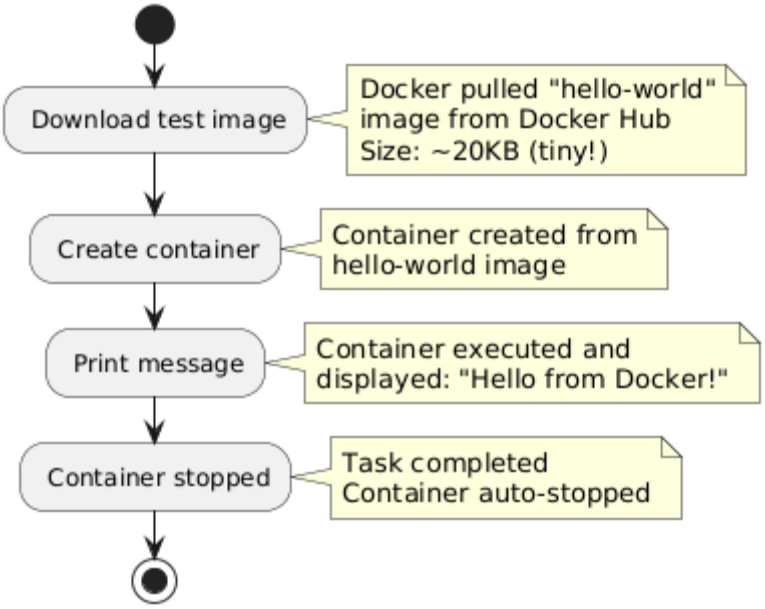
Step 2: Ran My First Container

To test that Docker was working correctly, I ran the “hello-world” test container.

Command Used:

sudo docker run hello-world

What This Did:



```
fadymohmed416@fadyabdelrahman-ca1-docker:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
Digest: sha256:6ae972d1ac530d897fd03238e11e16157498834b49574f21a0c7d0f51b58f84d
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Figure 3 Confirmed Docker was working properly

Step 3: Started the Docker Tutorial Container

Docker provides an interactive tutorial that runs in a container. I started it to access the step-by-step guide.

Command Used:

sudo docker run -d -p 80:80 docker/getting-started

Explanation:

- -d = Run in detached mode (background)
- -p 80:80 = Map port 80 on my server to port 80 in the container
- docker/getting-started = The name of the tutorial image

```
fadymohmed416@fadyabdelrahman-ca1-docker:~$ sudo docker run -d -p 80:80 docker/getting-started
Unable to find image 'docker/getting-started:latest' locally
latest: Pulling from docker/getting-started
c158987b0551: Pull complete
1e35f6679fab: Pull complete
cb9626c74200: Pull complete
b6334b6ace34: Pull complete
f1d1c9928c82: Pull complete
9b6f639ec6ea: Pull complete
ee68d3549ec8: Pull complete
33e0cbbb4673: Pull complete
4f7e34c2de10: Pull complete
Digest: sha256:d79336f4812b6547a53e735480dde67f8f8f7071b414fbd9297609ffb989abc1
Status: Downloaded newer image for docker/getting-started:latest
a953975d4b08240f4e65f1ccbfd7e95585744bf04ec286ab444228ea50619299
fadymohmed416@fadyabdelrahman-ca1-docker:~$ sudo docker ps
CONTAINER ID   IMAGE                     COMMAND                  CREATED         STATUS         PORTS                                     NAMES
a953975d4b08   docker/getting-started    "/docker-entrypoint.…"   8 seconds ago   Up 7 seconds   0.0.0.0:80->80/tcp, [::]:80->80/tcp   distracted_buck
fadymohmed416@fadyabdelrahman-ca1-docker:~$
```

Figure 4 Getting-Started-Browser

Access the Tutorial: I could access the tutorial at: http://35.246.71.231

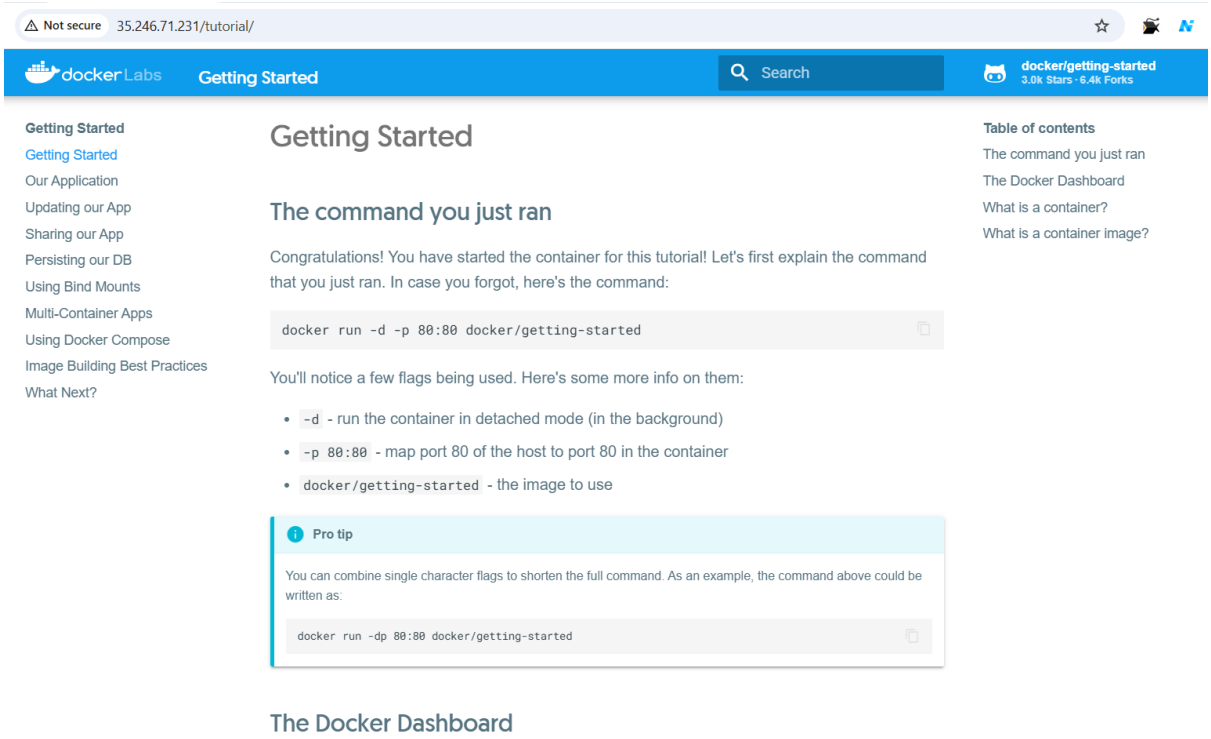


Figure 5 Tutorial Homepage

Part 2: Building My First Container

What I Did

Step 1: Downloaded the Sample Application

I cloned the Docker tutorial repository which contains a sample Node.js todo application.

Command Used:

```
git clone https://github.com/docker/getting-started.git
cd getting-started/app
```

```
fadymohmed416@fadyabdelrahman-ca1-docker:~$ git clone https://github.com/docker/getting-started.git
Cloning into 'getting-started'...
remote: Enumerating objects: 987, done.
remote: Total 987 (delta 0), reused 0 (delta 0), pack-reused 987 (from 1)
Receiving objects: 100% (987/987), 5.23 MiB | 23.78 MiB/s, done.
Resolving deltas: 100% (533/533), done.
fadymohmed416@fadyabdelrahman-ca1-docker:~$ cd getting-started/app
fadymohmed416@fadyabdelrahman-ca1-docker:~/getting-started/app$ ls
package.json  spec  src  yarn.lock
fadymohmed416@fadyabdelrahman-ca1-docker:~/getting-started/app$
```

Figure 6 App Folder Structure

What I Got

#	Description
1	A complete Node.js application with frontend and backend
2	Package dependencies defined in package.json
3	Application source code in the src/ folder

Step 2: Created a Dockerfile

A Dockerfile is like a recipe that tells Docker how to build a container image. I created this file in the app folder.

Dockerfile Content:

```
# syntax=docker/dockerfile:1
FROM node:18-alpine
WORKDIR /app
COPY. .
RUN yarn install --production
CMD ["node", "src/index.js"]
EXPOSE 3000
```

What Each Line Means

#	Line / Command	Meaning
1	`FROM node:18-alpine`	Start with a lightweight Node.js base image
2	`WORKDIR /app`	Set the working directory inside the container to /app
3	`COPY . .`	Copy all files from my computer into the container
4	`RUN yarn install --production`	Install the application dependencies
5	`CMD ["node", "src/index.js"]`	Command to run when container starts
6	`EXPOSE 3000`	Document that the app listens on port 3000

```
fadymohmed416@fadyabdelrahman-ca1-docker:~/getting-started/app$ nano Dockerfile
fadymohmed416@fadyabdelrahman-ca1-docker:~/getting-started/app$ ls
Dockerfile  package.json  spec  src  yarn.lock
fadymohmed416@fadyabdelrahman-ca1-docker:~/getting-started/app$
```

Figure 7 Dockerfile Created

Step 3: Built the Docker Image

I used the Dockerfile to build a container image.

Command Used:

sudo docker build -t getting-started .

Explanation

#	Command / Flag	Explanation
1	`docker build`	Command to build an image
2	`-t getting-started`	Tag (name) the image "getting-started"
3	`.`	Use the current directory (which contains the Dockerfile)

```
fadymohmed416@fadyabdelrahman-ca1-docker:~/getting-started/app$ sudo docker build -t getting-started
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  4.626MB
Step 1/6 : FROM node:18-alpine
18-alpine: Pulling from library/node
f18232174bc9: Pulling fs layer
dd71dde834b5: Pulling fs layer
1e5a4c89cee5: Pulling fs layer
25ff2da83641: Pulling fs layer
25ff2da83641: Waiting
1e5a4c89cee5: Verifying Checksum
1e5a4c89cee5: Download complete
f18232174bc9: Verifying Checksum
f18232174bc9: Download complete
f18232174bc9: Pull complete
25ff2da83641: Verifying Checksum
25ff2da83641: Download complete
dd71dde834b5: Verifying Checksum
dd71dde834b5: Download complete
dd71dde834b5: Pull complete
1e5a4c89cee5: Pull complete
25ff2da83641: Pull complete
Digest: sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
```

```
Status: Downloaded newer image for node:18-alpine
 ---> ee77c6cd7c18
Step 2/6 : WORKDIR /app
 ---> Running in 23326610ffc6
 ---> Removed intermediate container 23326610ffc6
 ---> e5b79480143d
Step 3/6 : COPY . .
 ---> d5d3ea34026e
Step 4/6 : RUN yarn install --production
 ---> Running in 65df58cff4d4
yarn install v1.22.22
[1/4] Resolving packages...
[2/4] Fetching packages...
[3/4] Linking dependencies...
[4/4] Building fresh packages...
Done in 11.75s.
 ---> Removed intermediate container 65df58cff4d4
 ---> 12bcbcc474b2
Step 5/6 : CMD ["node", "src/index.js"]
 ---> Running in 176f85f5e233
 ---> Removed intermediate container 176f85f5e233
 ---> e857ccc96d58
Step 6/6 : EXPOSE 3000
 ---> Running in 6c97bf6347ec
 ---> Removed intermediate container 6c97bf6347ec
 ---> 1c5d32719a34
Successfully built 1c5d32719a34
Successfully tagged getting-started:latest
```

What Happened:

1. Docker read my Dockerfile line by line
2. For each instruction, it created a layer
3. It downloaded the Node.js base image
4. It installed my application dependencies
5. It created a final image named “getting-started”

A new Docker image was created and stored locally.

Step 4: Verified the Image

I checked that my image was created successfully.

Command Used:

sudo docker images

Result: I could see “getting-started” in the list of images.

```
fadymohmed416@fadyabdelrahman-ca1-docker:~/getting-started/app$ sudo docker images
REPOSITORY                TAG         IMAGE ID       CREATED              SIZE
getting-started           latest      1c5d32719a34   About a minute ago   217MB
hello-world               latest      e2ac70e7319a   8 hours ago          10.1kB
node                      18-alpine   ee77c6cd7c18   12 months ago        127MB
docker/getting-started    latest      3e4394f6b72f   3 years ago          47MB
```

Figure 8 Image List

Step 5: Ran the Container

I created and started a container from my new image.

Command Used:

sudo docker run -dp 3000:3000 getting-started

Explanation

#	Flag / Command	Explanation
1	`-d`	Run in background (detached mode)
2	`-p 3000:3000`	Map port 3000 on host to port 3000 in container
3	`getting-started`	The image to use

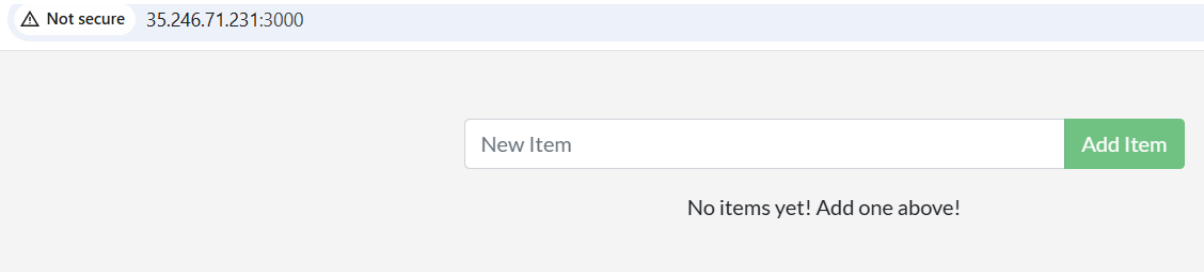


Figure 9 Todo App Running

Step 6: Verified Container Status

I checked that my container was running.

Command Used:

sudo docker ps

Result: I could see my container in the list with status “Up” and port mapping visible.

```
fadymohmed416@fadyabdelrahman-ca1-docker:~/getting-started/app$ sudo docker run -dp 3000:3000 getting-started
6862c99f9feeece69795d617bcff85778b3c6fdb5caf71d841d2316c9e55bceb
fadymohmed416@fadyabdelrahman-ca1-docker:~/getting-started/app$ sudo docker ps
CONTAINER ID   IMAGE                     COMMAND                  CREATED          STATUS          PORTS                                       NAMES
6862c99f9fee   getting-started           "docker-entrypoint.s…"   12 seconds ago   Up 11 seconds   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp   naughty_lederberg
a953975d4b08   docker/getting-started    "/docker-entrypoint.…"   13 minutes ago   Up 13 minutes   0.0.0.0:80->80/tcp, [::]:80->80/tcp           distracted_buck
```

Figure 10 Container Running

Part 3: Updating the Application

What I Did

Step 1: Modified the Source Code

Here made a small change to the application to learn how to update containerized apps.

What I Changed

#	Action	Details
1	Opened the file	`src/static/js/app.js`
2	Found the text	`No items yet! Add one above!`
3	Changed it to	`You have no todo items yet! Add one above!`

```
return (
    <React.Fragment>
        <AddItemForm onNewItem={onNewItem} />
        {items.length === 0 && (
        // Old text: <p className="text-center"> You have no todo items yet! Add one above! </p>
        // New text updated by Fady
          <p className="text-center">Ready to stay organized?Add your first todo item now.</p>
        )}
```

Figure 11 Code Modified

Step 2: Rebuilt the Image

After changing the code, I needed to rebuild the Docker image.

Command Used:

sudo docker build -t getting-started .

What Happened

#	Event	Details
1	Rebuilt the image	Docker rebuilt the image with my code changes
2	Faster build	The building was faster this time because Docker cached unchanged layers
3	Partials rebuild	Only the layers affected by my code change were rebuilt

Step 3: Removed the Old Container

Here I couldn't start a new container on port 3000 while the old one was still running, so I removed it.

Commands Used:

```
# Find container ID
sudo docker ps

# Stop and remove the container
sudo docker rm -f <container-id>
```

Step 4: Started the Updated Container

I started a new container using the updated image.

Command Used:

```
sudo docker run -dp 3000:3000 getting-started
```

Result: When I refreshed the browser, I saw my updated message!

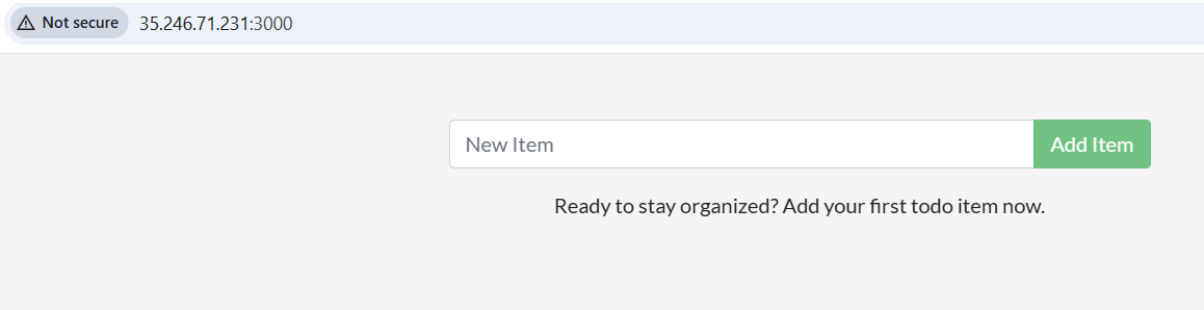


Figure 12 Updated App

Part 4: Sharing via Docker Hub

What I Did

Step 1: Created Docker Hub Account

Here I signed up for a free account at https://hub.docker.com to share my container image publicly.

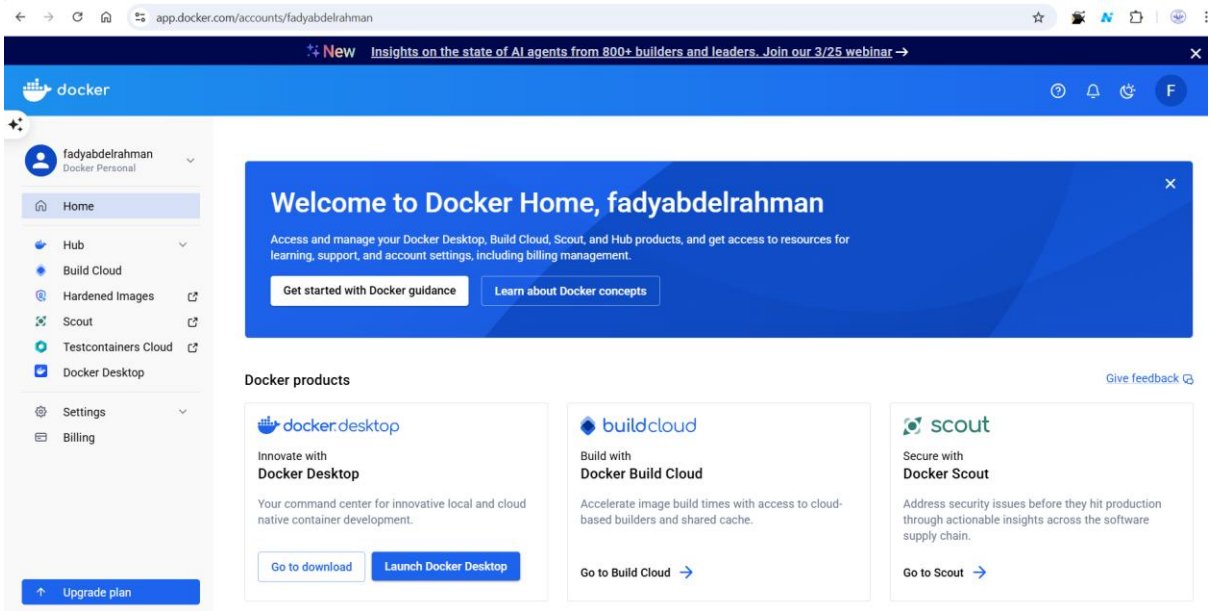


Figure 13 Docker Hub Dashboard

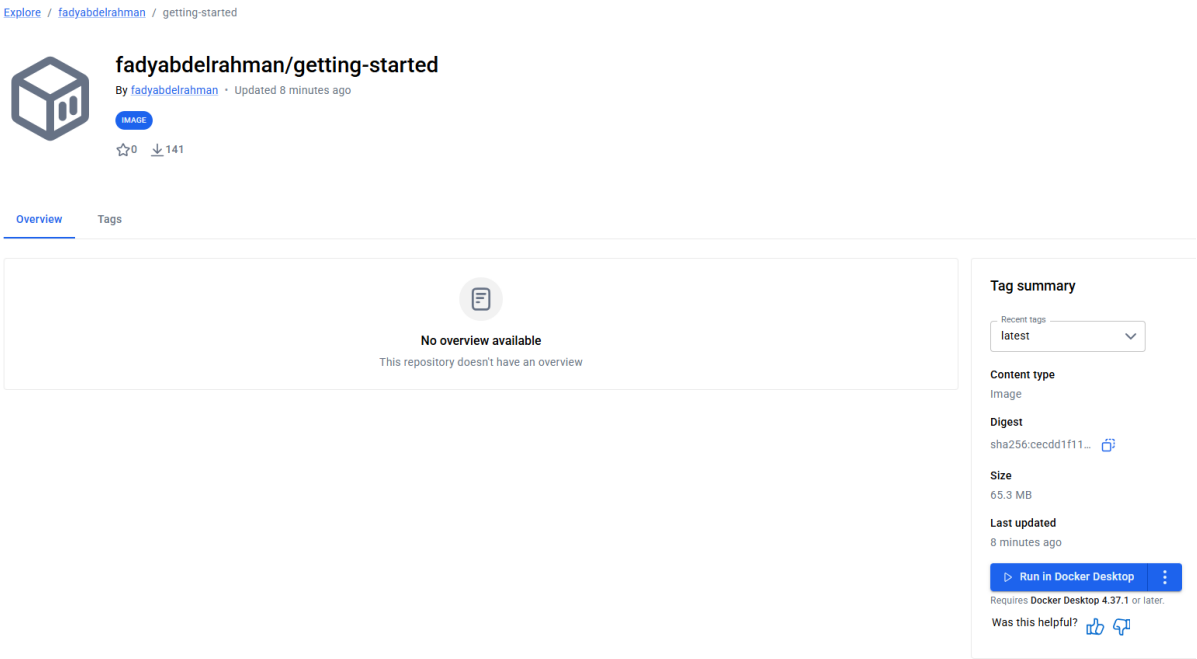


Figure 14 Successful push of the Docker image to Docker Hub repository

Step 2: Created a Repository

On Docker Hub, I created a new repository named “getting-started” to store my image.

Step 3: Logged In to Docker Hub

I authenticated my local Docker installation with Docker Hub.

Command Used:

sudo docker login

What I Entered

#	Field	Description
1	Docker Hub username	My Docker Hub account username
2	Docker Hub password	My Docker Hub account password

```
fadymohmed416@fadyabdelrahman-ca1-docker:~/getting-started/app$ sudo docker login -u fadyabdelrahman

i Info → A Personal Access Token (PAT) can be used instead.
         To create a PAT, visit https://app.docker.com/settings


Password:

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
```

Figure 15 Docker Login

Step 4: Tagged the Image

Before pushing to Docker Hub, I needed to tag my image with my username.

Command Used:

sudo docker tag getting-started fadyabdelrahman/getting-started

Explanation

#	Command / Part	Explanation
1	`docker tag`	Command to create a new tag for an image
2	`getting-started`	My local image name
3	`fadyabdelrahman/getting-started`	New name with Docker Hub username

Step 5: Pushed to Docker Hub

I uploaded my image to Docker Hub so others can download and use it.

Command Used:

sudo docker push fadyabdelrahman/getting-started

What Happened

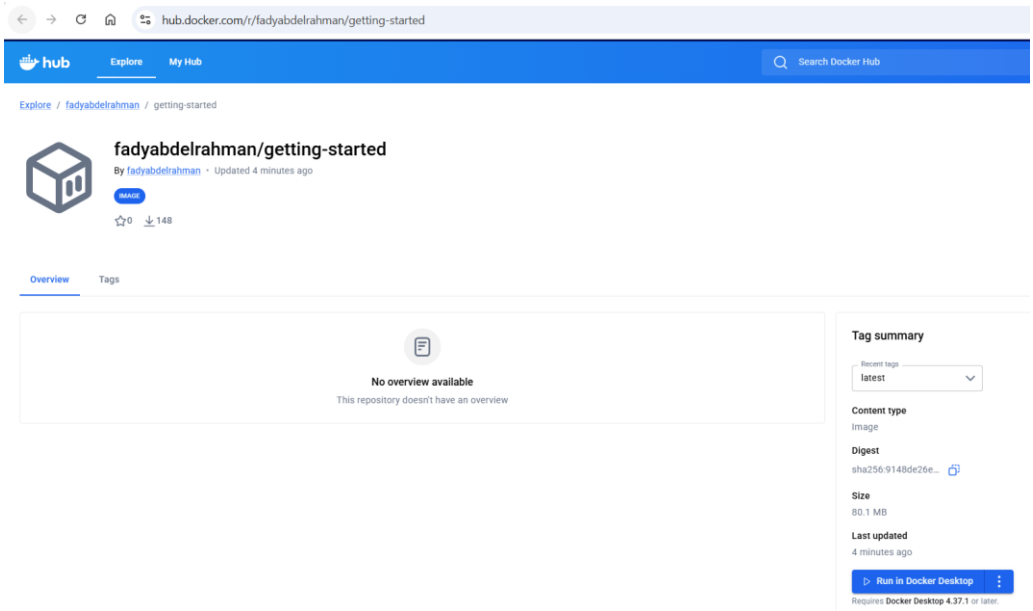
#	Event	Details
1	Layers uploaded	Docker uploaded each layer of my image
2	Image published	The image became publicly available on Docker Hub
3	Available to all	Anyone can now run: docker pull fadyabdelrahman/getting-started

```
fadymohmed416@fadyabdelrahman-ca1-docker:~/getting-started/app$ sudo docker push fadyabdelrahman/getting-started
Using default tag: latest
The push refers to repository [docker.io/fadyabdelrahman/getting-started]
4f833b970f2f: Layer already exists
0cafc8866abe: Layer already exists
e7fb6c4eecdf: Layer already exists
82140d9a70a7: Layer already exists
f3b40b0cdb1c: Layer already exists
0b1f26057bd0: Layer already exists
08000c18d16d: Layer already exists
latest: digest: sha256:9148de26e8cea1087e57a7054773e601c97d3cc5c2494c62ab45ad4312e5e9ff size: 1787
```

Figure 16 Image successfully pushed to Docker Hub.

Step 6: Verified on Docker Hub

I checked Docker Hub website to confirm my image was available. https://hub.docker.com/r/fadyabdelrahman/getting-started



Verified on Docker Hub

Part 5: Data Persistence with Volumes

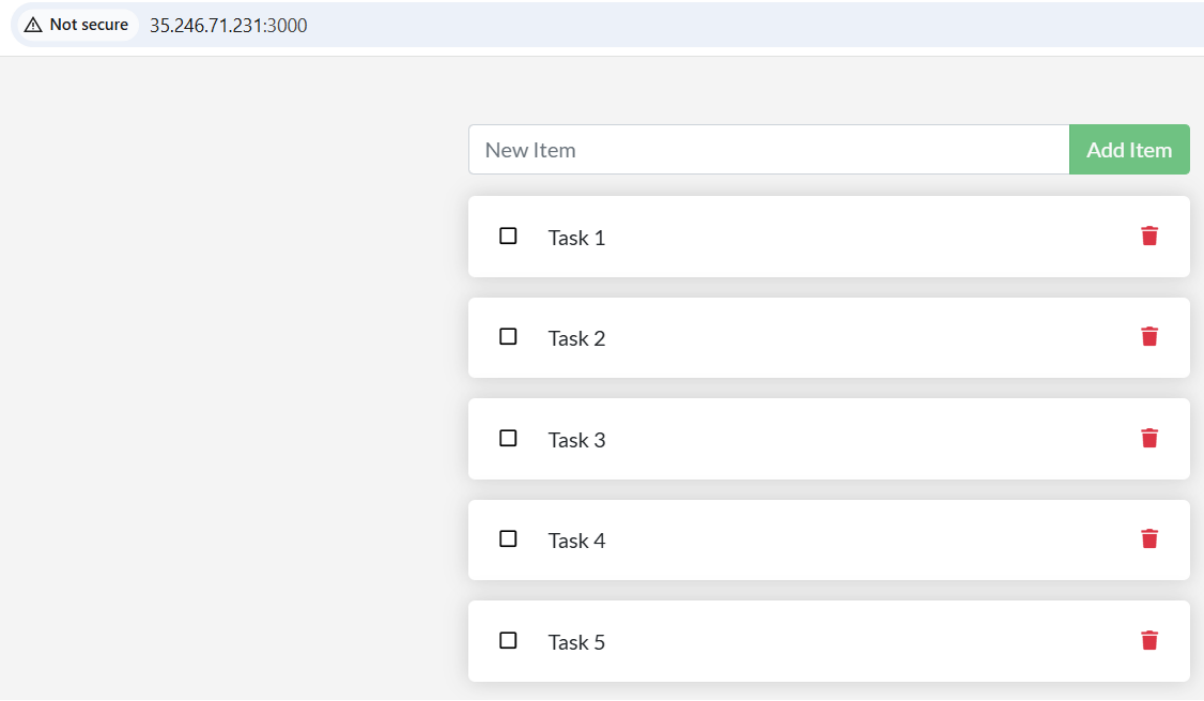
The Problem: Data Loss

Step 1: Demonstrated Data Loss

I discovered an important problem with containers: they are ephemeral (temporary).

What I Did

Step	Action
1	Added several todo items to my application
2	Noted the container ID
3	Stopped and removed the container
4	Started a new container
5	Refreshed the browser
Result	**All my todo items were gone!**



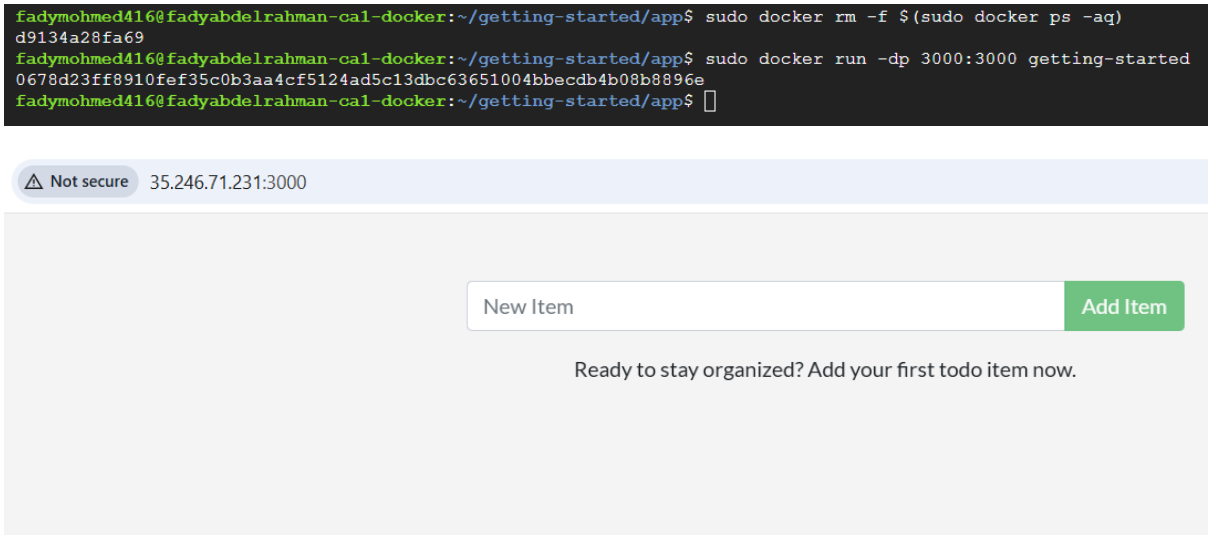


Figure 17 Todos Before Restart Todos Lost After Restart

Why This Happened:

- Containers store data inside themselves
- When a container is removed, all its data is deleted
- Each new container starts with a clean slate

The Solution: Docker Volumes

Step 2: Created a Docker Volume

Volumes are storage areas that exist outside containers and persist data.

Command Used:

sudo docker volume create todo-db

Verification:

sudo docker volume ls

Result: A new volume named “todo-db” was created.

Step 3: Ran Container with Volume

I started a new container and attached the volume to it.

Command Used:

sudo docker run -dp 3000:3000 -v todo-db:/etc/todos getting-started

Explanation

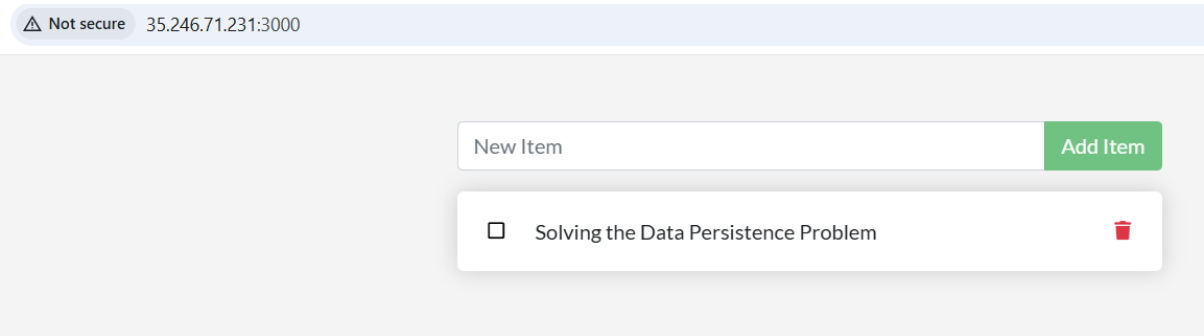
#	Command / Part	Explanation
1	`-v todo-db:/etc/todos`	Mount volume "todo-db" to "/etc/todos" in container
2	`/etc/todos/todo.db`	The app stores its data at this path (SQLite database file)
3	`Volume storage`	Now the data is stored in the volume, not in the container

Step 4: Tested Data Persistence

I tested if data now survives container restarts.

What I Did:

Step	Action
1	Added several todo items
2	Stopped and removed the container
3	Started a new container with the same volume attached
4	Refreshed the browser
Result	**All my todo items were still there!**




```
fadymohmed416@fadyabdelrahman-ca1-docker:~/getting-started/app$ sudo docker volume create todo-db
todo-db
fadymohmed416@fadyabdelrahman-ca1-docker:~/getting-started/app$ sudo docker volume ls
DRIVER    VOLUME NAME
local     todo-db
fadymohmed416@fadyabdelrahman-ca1-docker:~/getting-started/app$ sudo docker rm -f $(sudo docker ps -aq)
b2cf334fbb28
fadymohmed416@fadyabdelrahman-ca1-docker:~/getting-started/app$ sudo docker run -dp 3000:3000 -v todo-db:/etc/todos getting-started
b30ce5d375aef43183acb9b3dd4515238cf30c8d279a3d49db4d2d063b7bf192
fadymohmed416@fadyabdelrahman-ca1-docker:~/getting-started/app$ sudo docker rm -f $(sudo docker ps -aq)
b30ce5d375ae
fadymohmed416@fadyabdelrahman-ca1-docker:~/getting-started/app$ sudo docker run -dp 3000:3000 -v todo-db:/etc/todos getting-started
0114a046b49e63f96c49e2cadfddeacd688502e9dcc0903716cc5fa01f0cf676
```

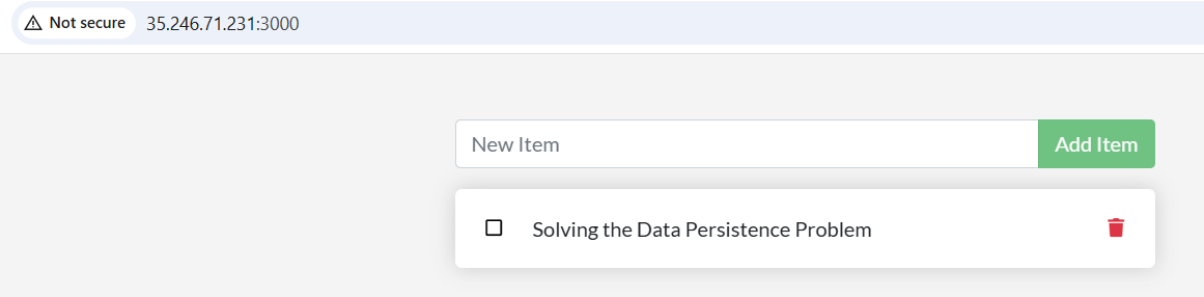


Figure 18 Data Persisted

Part 6: Multi-Container Applications (Most Important)

Understanding the Architecture

The Scenario:

My todo app currently uses SQLite (a single-file database). For production environments, I want to use MySQL (a proper database server). This means I need:

- **Container 1:** Todo application (Node.js)

- **Container 2:** MySQL database server

Why Separate Containers?

#	Reason	Benefit
1	**Independent scaling**	I can scale each component independently
2	**Safe database updates**	I can update the database without touching the app
3	**Process isolation**	Each process runs in isolation (better security)
4	**Industry best practice**	This is industry best practice (microservices architecture)

Architecture Diagram:

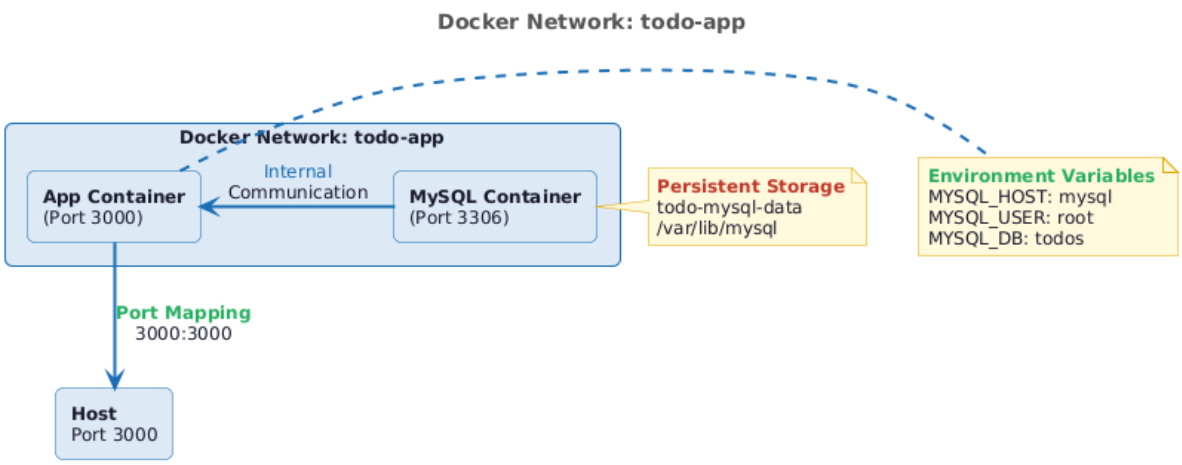


Figure 19 Multi-Container Architecture

Manual Multi-Container Setup

Step 1: Created a Docker Network

Containers need a network to communicate with each other.

Command Used:

sudo docker network create todo-app

Verification:

docker network ls

```
fadymohmed416@fadyabdelrahman-ca1-docker:~/getting-started/app$ sudo docker network create todo-app
Error response from daemon: network with name todo-app already exists
fadymohmed416@fadyabdelrahman-ca1-docker:~/getting-started/app$ sudo docker network ls
NETWORK ID     NAME       DRIVER    SCOPE
68905b99c199   bridge     bridge    local
78b9ef46bf01   host       host      local
d521cfdc96c6   none       null      local
c3435e320e38   todo-app   bridge    local
```

Figure 20 Network Created

Step 2: Started MySQL Container

I started a MySQL database container on the network.

Command Used:

```
sudo docker run -d \
 --network todo-app --network-alias mysql \
 -v todo-mysql-data:/var/lib/mysql \
 -e MYSQL_ROOT_PASSWORD=fady \
 -e MYSQL_DATABASE=todos \
 mysql:8.0
```

Explanation

#	Flag / Command	Explanation
1	`--network todo-app`	Connect to my network
2	`--network-alias mysql`	Give it hostname "mysql" on the network
3	`-v todo-mysql-data:/var/lib/mysql`	Persist MySQL data in a volume
4	`-e MYSQL_ROOT_PASSWORD=fady`	Set root password
5	`-e MYSQL_DATABASE=todos`	Create a database named "todos"
6	`mysql:8.0`	Use MySQL version 8.0 image

Step 3: Verified Both Containers Running

Command Used:

```
sudo docker ps
```

Both the MySQL container and my previous containers.

Step 4: Connected to MySQL

Here I tested that MySQL was working by connecting to it.

Command Used:

```
sudo docker exec -it <mysql-container-id> mysql -u root -p
```

Inside MySQL:

```
SHOW DATABASES;
USE todos;
SHOW TABLES;
EXIT;
```

```
fadymohmed416@fadyabdelrahman-ca1-docker:~$ sudo docker exec -it $(sudo docker ps -q --filter "ancestor=mysql:8.0") mysql -u root -pfady
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.45 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| todos              |
+--------------------+
5 rows in set (0.01 sec)

mysql> USE todos;
Database changed
mysql> SHOW TABLES;
Empty set (0.01 sec)
```

Figure 21 MySQL Connected

Step 5: Connected App to MySQL

I stopped the old app container and started a new one configured to use MySQL.

Commands Used:

```
# Stop old app container
sudo docker rm -f <app-container-id>

# Start app with MySQL connection
sudo docker run -dp 3000:3000 \
  --network todo-app \
  -e MYSQL_HOST=mysql \
  -e MYSQL_USER=root \
  -e MYSQL_PASSWORD=fady\
  -e MYSQL_DB=todos \
  getting-started
```

Explanation:

- --network todo-app = Connect to same network as MySQL

- -e MYSQL_HOST=mysql = Connect to hostname “mysql” (my network alias)

- Other -e flags = Database credentials

```
fadymohmed416@fadyabdelrahman-ca1-docker:~$ sudo docker run -dp 3000:3000 \
  --network todo-app \
  -e MYSQL_HOST=mysql \
  -e MYSQL_USER=root \
  -e MYSQL_PASSWORD=fady \
  -e MYSQL_DB=todos \
  getting-started
0cc76d6114400111a9f3446b4b16d7253d357d2b4f7e57465bd3c9a092fc0dc2
```

Figure 22 App with MySQL

Step 6: Verified App Uses MySQL

I added todos and checked they were stored in MySQL.

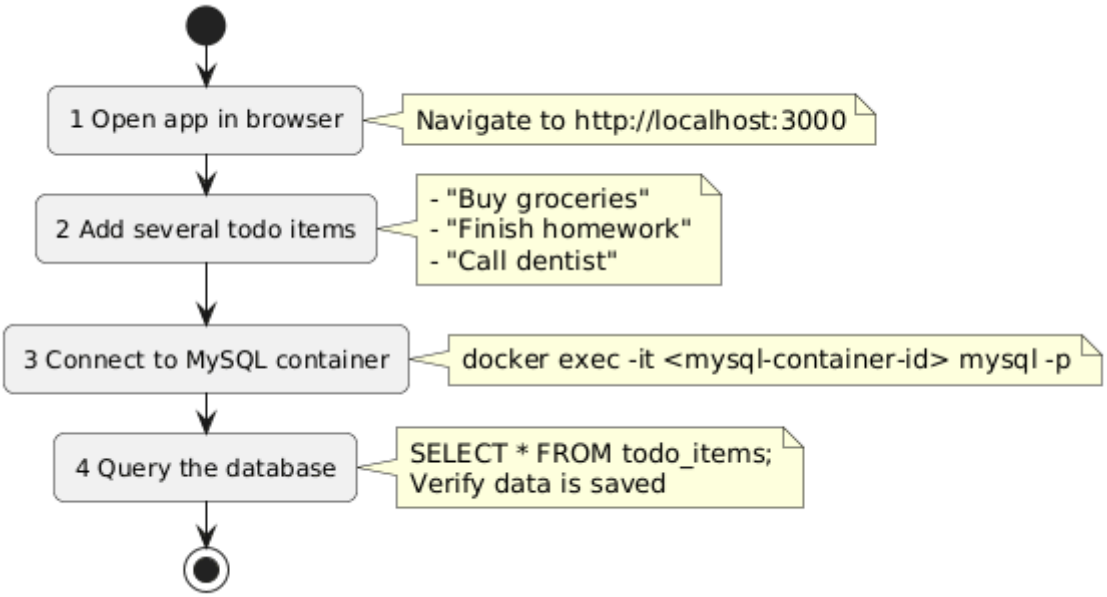


Figure 23 MySQL - User Journey

MySQL Query:

SELECT * **FROM** todo_items;

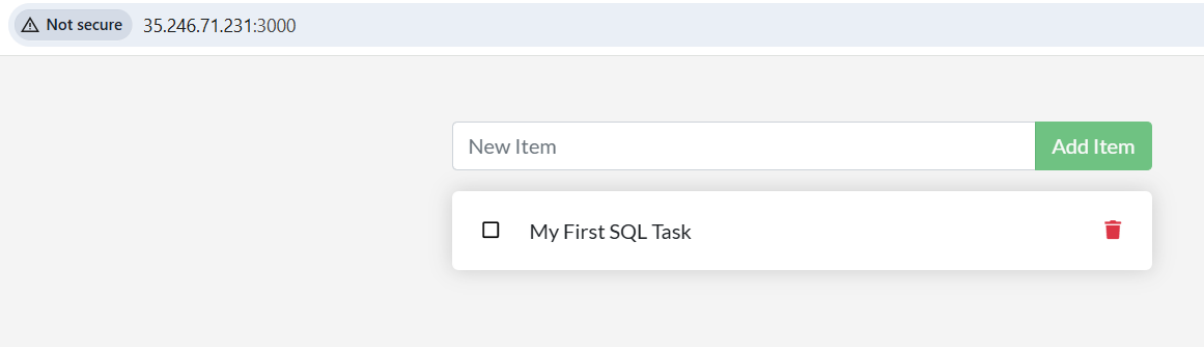


Figure 24 Browser with Todos MySQL Data added

```
fadymohmed416@fadyabdelrahman-ca1-docker:~$ sudo docker exec -it $(sudo docker ps -q --filter "ancestor=mysql:8.0") mysql -u root -pfady
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.45 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE todos;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SELECT * FROM todo_items;
+--------------------------------------+------------------+-----------+
| id                                   | name             | completed |
+--------------------------------------+------------------+-----------+
| 08d3742d-9e23-4b50-8f4d-744568a19db2 | My First SQL Task |         0 |
+--------------------------------------+------------------+-----------+
1 row in set (0.00 sec)

mysql>
```

Figure 25 MYSQL DATA

Docker Compose (Automated Multi-Container)

The Problem with Manual Setup

#	Issue	Description
1	**Long commands**	Long commands with many flags
2	**Easy to forget**	Easy to forget options
3	**Hard to share**	Hard to share with team
4	**Manual management**	Manual network and volume management

The Solution: Docker Compose

#	Feature	Benefit
1	**Single YAML file**	Define all containers in one YAML file
2	**One command**	Start everything with one command
3	**Version control ready**	Easy to version control and share

Step 7: Created docker-compose.yml

I created a configuration file that defines both containers.

File Location: Project root (parent of app folder)

File Content:

```yaml
services:
  app:
    image: getting-started
    ports:
      - 3000:3000
    environment:
      MYSQL_HOST: mysql
      MYSQL_USER: root
      MYSQL_PASSWORD: fady
      MYSQL_DB: todos
    depends_on:
      - mysql
    networks:
      - todo-app

  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: fady
      MYSQL_DATABASE: todos
    volumes:
      - todo-mysql-data:/var/lib/mysql
    networks:
      - todo-app

networks:
  todo-app:

volumes:
  todo-mysql-data:
```

What This Defines

#	Component	Details
1	**Services**	Two services: "app" and "mysql"
2	**Network**	"todo-app" (automatically created)
3	**Volume**	"todo-mysql-data" (automatically created)
4	**Dependencies**	App waits for MySQL to start first
5	**Environment variables**	Database connection settings

```
fadymohmed416@fadyabdelrahman-ca1-docker:~$ cat docker-compose.yml
services:
  app:
    image: getting-started
    ports:
      - 3000:3000
    environment:
      MYSQL_HOST: mysql
      MYSQL_USER: root
      MYSQL_PASSWORD: fady
      MYSQL_DB: todos
    depends_on:
      - mysql
    networks:
      - todo-app
  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: fady
      MYSQL_DATABASE: todos
    volumes:
      - todo-mysql-data:/var/lib/mysql
    networks:
      - todo-app

networks:
  todo-app:

volumes:
  todo-mysql-data:
```

Figure 26 Compose File

Step 8: Stopped Manual Containers

Before using Docker Compose, I cleaned up the manually created containers.

Command Used:

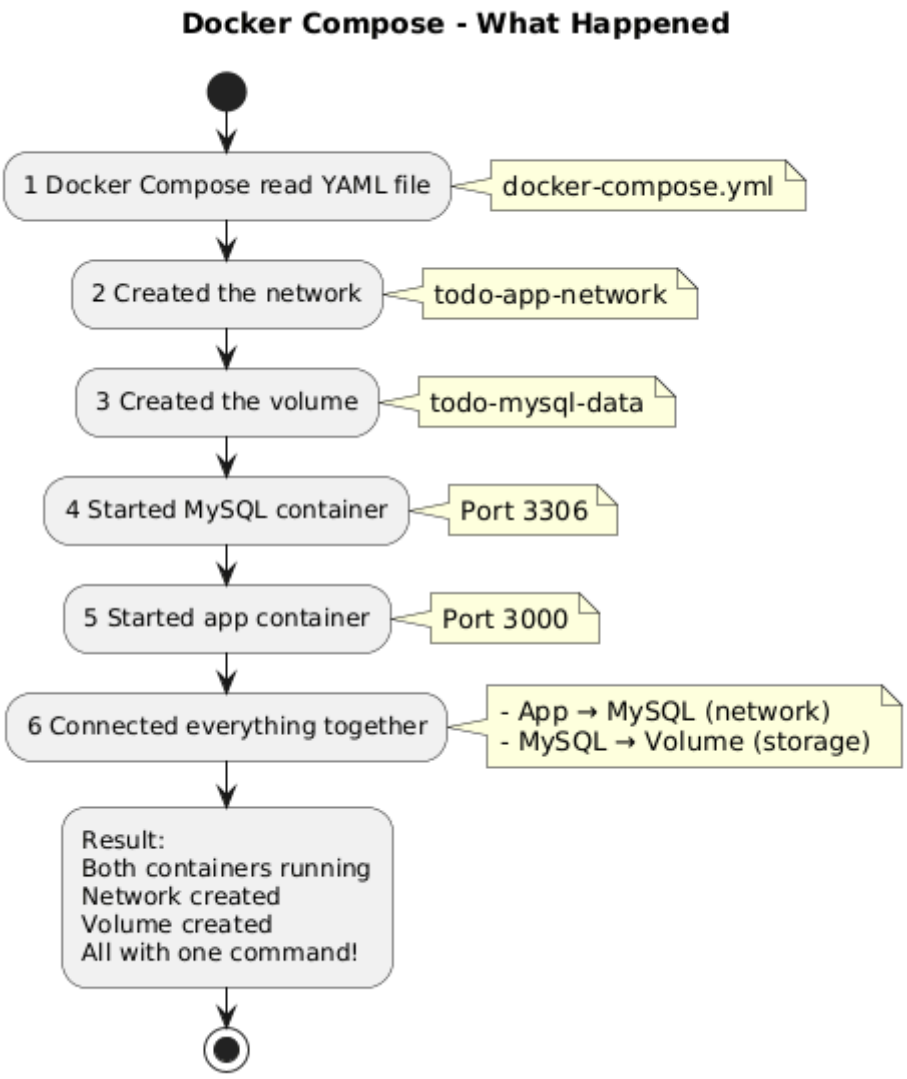
sudo docker ps -q | xargs sudo docker rm -f

Step 9: Started with Docker Compose

I used one simple command to start everything.

Command Used:

sudo docker compose up -d



Step 10: Verified Compose Deployment

Command Used:

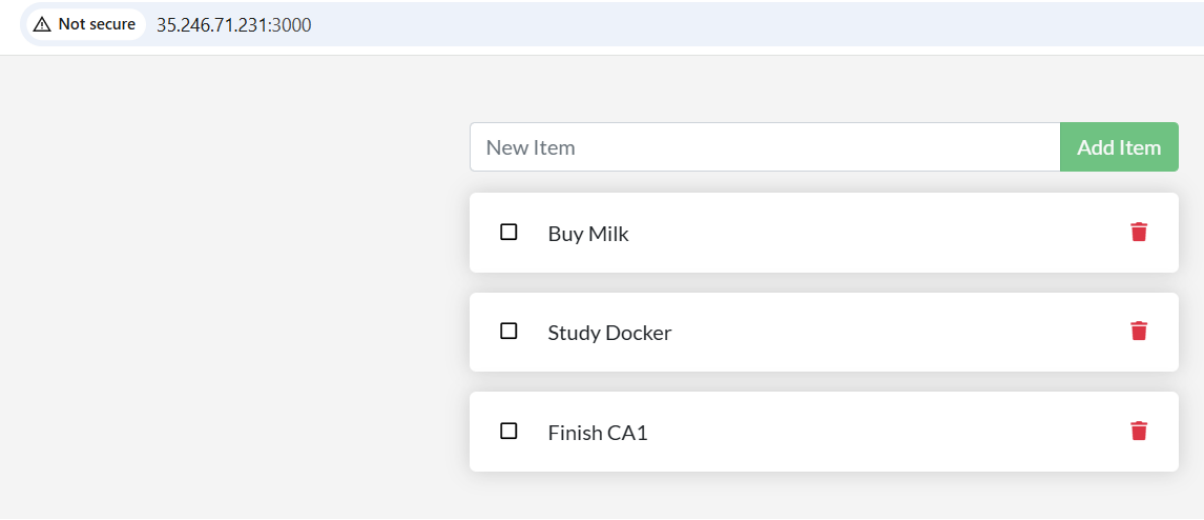
sudo docker compose ps

```
fadymohmed416@fadyabdelrahman-ca1-docker:~/getting-started$ sudo docker compose ps
WARN[0000] /home/fadymohmed416/getting-started/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME                    IMAGE                   COMMAND                  SERVICE   CREATED          STATUS          PORTS
getting-started-docs-1   getting-started-docs   "mkdocs serve -a 0.0…"   docs      14 seconds ago   Up 13 seconds   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
fadymohmed416@fadyabdelrahman-ca1-docker:~/getting-started$
```

Figure 27 Compose PS

Step 11: Tested the Application

I opened the browser and added todos to verify everything works.



Step 12: Viewed Logs

I checked the logs to see what's happening inside the containers.

Command Used:

sudo docker compose logs

Step 13: Stopped and Restarted

I tested that data persists across Docker Compose restarts.

Commands Used:

 # Stop everything

sudo docker compose down

 # Start again

sudo docker compose up -d

When I checked the browser, all my todos were still there!

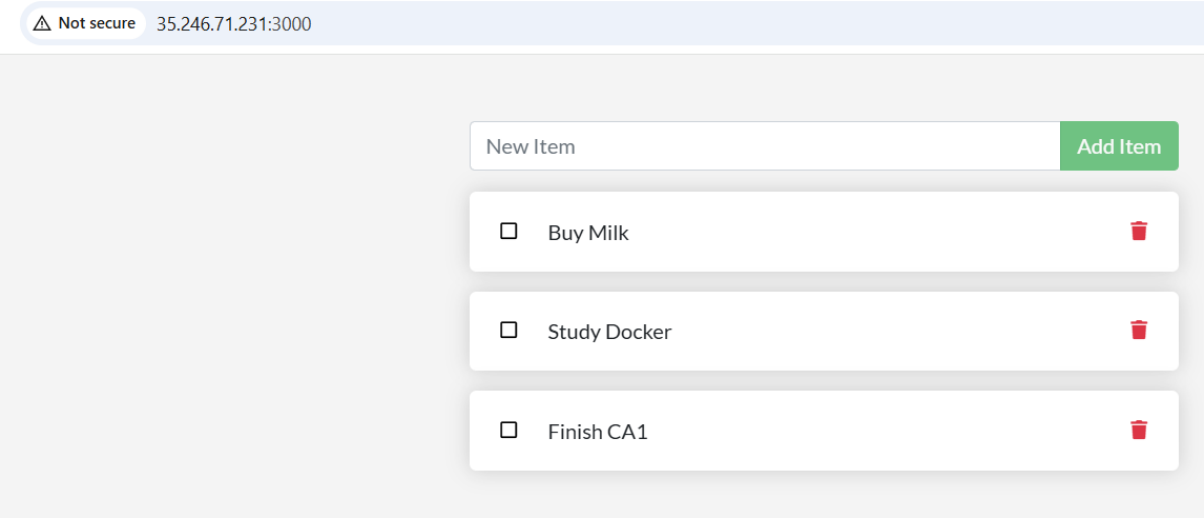


Figure 28 Compose App Running

Conclusion

This assignment successfully demonstrated comprehensive Docker proficiency through systematic completion of the official "Getting Started" tutorial. I progressed from basic installation to advanced multi-container orchestration, gaining practical experience with containerization, data persistence, and infrastructure automation.

Key Achievements:

- Installation: I configured Docker on Ubuntu 22.04 and successfully ran the tutorial container.
- Practical Application: I managed to access the application through the server's IP address and understood how port mapping works.
- Core Concepts: The project helped me understand important topics like Dockerfiles, Images, and Volumes for saving data.
- Cloud Readiness: These skills are essential for modern cloud infrastructure and professional development.

References

1. *Docker Inc. (2026). Docker Documentation. Retrieved from https://docs.docker.com*

2. *Docker Inc. (2026). Get Started with Docker. Retrieved from https://docs.docker.com/get-started/*

3. *Docker Inc. (2026). Docker Compose Overview. Retrieved from https://docs.docker.com/compose/*

4. *Docker Inc. (2026). Dockerfile Reference. Retrieved from https://docs.docker.com/engine/reference/builder/*

5. *Docker Inc. (2026). Getting Started Tutorial Repository. GitHub. Retrieved from https://github.com/docker/getting-started*

6. *Docker Inc. (2026). Docker Hub. Retrieved from https://hub.docker.com*

7. *Poulton, N. (2023). The Docker Book: Containerization is the new virtualization. Docker Deep Dive.*

8. *Turnbull, J. (2024). Docker: Up & Running. O'Reilly Media.*

9. *Ubuntu. (2026). Ubuntu 22.04 LTS Documentation. Retrieved from https://ubuntu.com/server/docs*

10. *Google Cloud. (2026). Compute Engine Documentation. Retrieved from https://cloud.google.com/compute/docs*

11. *Docker Documentation. (2024). Docker overview. Retrieved from https://docs.docker.com/get-started/overview/*

12. *Google Cloud. (2024). Compute Engine documentation. Retrieved from https://cloud.google.com/compute/docs*

13. *Kubernetes Documentation. (2024). What is Kubernetes?. Retrieved from https://kubernetes.io/docs/concepts/overview/what-is-kubernetes/*

14. *Red Hat. (2024). What is a private cloud? Retrieved from https://www.redhat.com/en/topics/cloud-computing/what-is-private-cloud*

15. *Abdelrahman, F. (2026). CA1 - Docker Tutorial Complete Guide. GitHub Repository. Retrieved from https://github.com/FadyAbdelrahman/CA1-Docker-Cloud-Infrastructure/tree/master*